In the Polar Bookshop system, we need to implement an event-driven solution to allow different applications to communicate with each other asynchronously while reducing their coupling. These are the requirements:

- When an order is accepted:
    - Order Service should notify interested consumers of the event.
    - Dispatcher Service should execute some logic to dispatch the order.
- When an order is dispatched:
    - Dispatcher Service should notify consumers interested in such an event.
    - Order Service should update the order status in the database.

If you paid attention, you probably noticed that the requirements don’t specify which applications Order Service should notify upon order creation. In our example, only the new Dispatcher Service application will be interested in those events. Still, more applications might subscribe to the order creation events in the future. The beauty of this design is that you can evolve a software system and add more applications without affecting the existing ones at all. For example, you could add a Mail Service that sends an email to users whenever an order they made has been accepted, and Order Service wouldn’t even be aware of it.

This type of interaction should be asynchronous and can be modeled with the pub/sub model. Figure 10.1 illustrates the interaction and describes three flows for accepting, dispatching, and updating an order. They are temporally decoupled and executed asynchronously. You will probably notice that the operations for persisting data into a database and for producing events have